

KeynetikPOS

Technical Documentation

Architecture, data layer, domain subsystems and build

Version 2.1.0 | Branch: feature/erp-tier4
Qt 6.11 · C++17 · SQLite · Cloudflare Workers
Current as of 19 July 2026

Contents

- 1** System overview
- 2** Toolchain and build
- 3** Architecture and ownership
- 4** The data layer
- 5** Schema and migrations
- 6** Money as a value type
- 7** The sale pipeline
- 8** Finance and accounting (Tier 4)
- 9** M-Pesa payment integration
- 10** Licensing and tiers
- 11** Theming and the UI system
- 12** Testing
- 13** Packaging and deployment
- 14** Known gaps and technical debt

1 System overview

KeynetikPOS is a Windows desktop point-of-sale and light-ERP application for the Kenyan retail market. It is a single-process Qt Widgets application over a local SQLite database, paired with a Cloudflare Worker that handles licensing and brokers M-Pesa payments.

Property	Value
Language / standard	C++17
UI framework	Qt 6.11 Widgets
Persistence	SQLite via Qt SQL, WAL journal mode
Backend	Cloudflare Worker + D1 (licensing, M-Pesa brokering)
Source size	~28,700 lines across 66 .cpp and 74 .h files
Test suites	7 (CTest)
Target market	Kenya — KSh currency, 16% VAT, M-Pesa, KRA eTIMS

1.1 Functional scope

- **Point of sale** — product grid, cart, split-tender checkout, discounts, refunds, receipt printing and emailing, barcode scanning.
- **Inventory** — catalogue CRUD, per-product reorder levels, stock adjustment audit trail, stock-take, low-stock alerting, valuation.
- **Purchasing** — suppliers, purchase orders, transactional goods receipt that updates stock and cost.
- **Customers** — store credit, loyalty points, purchase history.
- **Reporting** — analytics dashboard, detailed and accountant reports, Z-reports, profit and loss.
- **Finance (Tier 4)** — general ledger, VAT returns, payroll with Kenyan statutory deductions, automatic journal posting.

1.2 Design principles visible in the code

- **Money is never a float.** A dedicated **Money** value type wraps integer cents; the compiler flags every boundary (section 6).
- **Sales are atomic.** Stock validation and decrement occur in the same transaction as the sale insert, so a sale cannot rest on a stock count that changed underneath it (section 7).
- **History is snapshotted.** Sale line items copy name, price and cost at sale time, because products are later renamed, repriced and retired.
- **Offline shops must not be locked out.** Licence activation is online-first with a local fallback (section 10).
- **Secrets stay server-side.** The till never holds Daraja credentials (section 9).

2 Toolchain and build

2.1 Environment

Component	Version / location
Qt	6.11.0, MinGW 64-bit — C:\Qt\6.11.0\mingw_64
Compiler	MinGW 13.1.0 — C:\Qt\Tools\mingw1310_64\bin
Build system	CMake, Qt Creator generated trees under build/
Installer	NSIS — C:\Program Files (x86)\NSIS
Qt modules	Widgets, Core, Sql, PrintSupport, Network, Concurrent, SerialPort
Windows libs	crypt32 (DPAPI, for credential encryption)

2.2 Building

```
$env:PATH = "C:\Qt\Tools\mingw1310_64\bin;C:\Qt\6.11.0\mingw_64\bin;" + $env:PATH
cmake --build "build\Desktop_Qt_6_11_0_MinGW_64_bit-Release"
ctest --test-dir "build\Desktop_Qt_6_11_0_MinGW_64_bit-Release" --output-on-failure
```

Changing the project version forces a full rebuild

`project(KeynetikPOS VERSION ...)` participates in CMake configuration, so bumping it reconfigures and recompiles the entire tree rather than building incrementally. Budget several minutes.

2.3 Compiler warnings

Warnings are enabled but not fatal — **-Wall -Wextra** on GCC, **/W4 /utf-8** on MSVC. **-Werror** is deliberately not set because pre-existing warnings remain. The MSVC **/utf-8** flag is required: the sources are UTF-8 without BOM, and without it MSVC reads string literals in the system code page and corrupts non-ASCII text.

3 Architecture and ownership

3.1 Layers

Layer	Responsibility	Representative files
Presentation	Qt Widgets dialogs and models. Holds no business rules beyond input validation.	mainwindow, paymentdialog, inventorydialog, cartmodel, productgridmodel
Services	Orchestration across repositories and UI-independent workflows.	checkoutservice, cartservice, inventorymanager, schedulemanager
Domain	Pure, testable computation with no I/O.	money, carttotals, salejournal, payroll, vat, discount
Data access	Eight repositories, each owning the SQL for one aggregate.	productrepository, salerepository, customerrepository, ...

Layer	Responsibility	Representative files
Infrastructure	Connection lifecycle, schema, migrations, backup, integrity.	database
Integrations	Outbound network and hardware.	mpesaclient, licensemanager, smtpclient, barcodereader, etims

3.2 The pure-function core

The most valuable design decision in the codebase is that the calculations which must be correct are pure functions taking values and returning values, with no database or widget dependency. **computeCartTotals()**, **computePayslip()**, **splitInclusive()** and the **build*Journal()** family are all of this shape. This is why they carry genuine unit tests while the UI does not — the tests exercise the logic directly rather than through a widget hierarchy.

3.3 Ownership and dependency injection

Database is not a singleton. **Database::instance()** was removed and there are now zero references to it in the codebase. **main()** constructs one **Database db**; for the process and injects it as a reference into **MainWindow**, which forwards it to every dialog, manager and service it constructs. Each consumer stores a **Database &m_db** member.

```
Database db;                                // main.cpp – the app owns the connection
if (!db.initialize()) { /* fatal */ }
db.backupIfDue();
MainWindow w(db);                          // injected by reference
```

This exists so tests can supply an isolated in-memory database via **configureForTesting()** rather than sharing process-global state.

Three singletons are retained deliberately

UserManager, **LicenseManager** and **ThemeManager** remain singletons because they model genuinely process-global state — one operator session, one licence, one theme. Unlike the database, they have no per-test isolation requirement. Blanket removal of the remaining **instance()** call sites was assessed as low return for high risk.

Where a service would otherwise reach for a singleton, the dependency is passed instead. **CheckoutService** takes an **OperatorContext** — a plain struct of username, full name and a logging callback — so it can be exercised with no user session at all.

4 The data layer

Database exposes eight lazy accessors returning references to repository instances it owns. Consumers call through the accessor and read errors from the same accessor.

```
if (!m_db.customers().addCustomer(c))
    show(m_db.customers().lastError());

const auto items = m_db.products().getAllProducts();
```

Each accessor returns a reference to a persistent member — never a temporary — so an error read immediately after a failed write is still valid.

Repository	Owns
ProductRepository	Catalogue CRUD, stock mutators, adjustStockWithLog, stock history, valuation.
SaleRepository	recordSale plus all sale reads — getAllSales, date ranges, line items, customer purchase history.
SalesAnalyticsRepository	Aggregate reads — totals by day and month, top sellers, payment totals by method, gross profit, profit and loss.
RefundRepository	processRefund and isRefunded. Cross-table and transactional; orchestrates SaleRepository and ProductRepository on the shared connection.
SupplierRepository	Supplier CRUD and deactivation.
PurchaseOrderRepository	PO CRUD and the transactional receivePurchaseOrder, which updates stock and cost and writes per-line audit rows.
ExpenseRepository	Expense categories, expenses, totals.
CustomerRepository	Customer CRUD, store credit adjustment, loyalty point redemption.

Accessors cache a copy of the connection handle

configureForTesting() swaps the underlying connection, so it calls **resetRepositories()** before removing the old database. Without that, the second test in a run would use a dangling handle and crash — a failure mode that presented as the first test always passing when run alone.

Database itself now holds only connection lifecycle, schema creation and migration, sample-data seeding, backup and integrity checking. **getLastError()** survives on it for those lifecycle paths alone.

5 Schema and migrations

5.1 Tables

The database file lives in the per-user AppData directory, keeping it out of read-only Program Files. Foreign keys are enabled, and `initialize()` sets `journal_mode=WAL` with a 5-second busy timeout.

Area	Tables
Catalogue & stock	products, categories, stock_adjustments
Sales	sales, sale_items, sale_payments, refunds
Parties	customers, suppliers
Purchasing	purchase_orders, purchase_order_items
Expenses	expenses, expense_categories
Ledger	gl_accounts, gl_entries, gl_lines
Payroll	employees, payslips, pay_runs, payroll_config
VAT	vat_config
Operations	users, user_activity_log, till_reconciliations, message_schedules
Meta	schema_meta

5.2 Versioned migrations

Schema evolution runs through a numbered migration runner. The version is stored as a row in `schema_meta`, and each step above the current version applies inside its own transaction so the DDL and the version bump commit together. Steps are written idempotently, so fresh and old databases converge on the same shape.

Never gate a migration on `PRAGMA user_version`

An earlier implementation tracked completion with `PRAGMA user_version`, which does **not** persist across close and reopen under WAL via the Qt SQLite driver. The value read back as zero on every launch, so the money migration re-ran each time and re-scaled every monetary value by 100 — silent, compounding data corruption. The fix was a marker row in a real table, written in the same transaction as the data change. A regression test pins this behaviour using a real file database rather than an in-memory one.

The money migration remains deliberately separate from the numbered sequence, gated by its own marker. Version-gating it would cause a legacy database reporting version zero to be re-scaled.

6 Money as a value type

Monetary amounts are a strong type wrapping `qint64` cents, not doubles.

```
class Money {
    static constexpr Money fromCents(qint64 cents);
    static Money fromMajor(double major);
    constexpr qint64 cents() const;
    constexpr double toMajor() const;    // cents / 100.0
    explicit constexpr Money(qint64 cents);    // private
};
```

Conversions are explicit in both directions. Because there is no implicit conversion to or from **double**, the compiler flags every boundary where a raw number meets a monetary value — which is precisely how the original migration was carried out safely across the domain structs, the database columns and the reporting code.

- Domain structs carry **Money**: product price and cost, all sale and line-item totals, cart discount, receipt fields, analytics metrics.
- Database money columns are **INTEGER**.
- Cart totals are computed entirely in integers — tax is a single **llround** of base times rate.
- UI dialogs keep **QDoubleSpinBox** internally and convert only at the getter boundary.

Why this was worth the migration

A characterisation test documents the original defect: with doubles, rounding 1.005 yielded 1.00 while 10.005 yielded 10.01 — the same half-cent input rounding in opposite directions depending on magnitude. Cash-drawer float amounts in the shift and payment managers remain doubles; that is a separate concept from the sales money path and is intentional.

7 The sale pipeline

7.1 Flow

- 1 The cashier builds a cart; **CartService** holds the state and **CartModel** presents it, clamping quantity edits to available stock.
- 2 **computeCartTotals()** derives subtotal, discount, tax and total from the cart and the business settings. Pure and unit-tested.
- 3 **PaymentDialog** collects tenders — cash, card, mobile money, store credit, loyalty redemption — and refuses to confirm until the entered amount covers the total.
- 4 **CheckoutService::finalizeSale()** builds a **SaleRequest** and calls **recordSale()**.
- 5 **recordSale()** re-validates stock, inserts the sale header and line items, and decrements stock — all inside one transaction.
- 6 On success the sale is posted to the general ledger, the receipt is printed if a printer is attached, and the operator action is logged.

7.2 Transactional guarantees

Validation and mutation share a transaction

Stock is re-checked inside the same transaction that decrements it. Any failure — including insufficient stock on a single line — rolls the whole sale back, leaving stock untouched. This is directly covered by `database_test` and `checkoutservice_test`.

Refunds are guarded symmetrically: **processRefund** checks **isRefunded()** *inside* the transaction, closing a window in which a sale could be refunded twice.

7.3 Tenders and split payment

A sale carries a vector of **SalePayment** entries of method, amount and reference. Cash absorbs change, so its recorded contribution is the remainder after the exact tenders rather than the amount handed over. Card and mobile-money tenders carry their reference — for M-Pesa, the ten-character receipt code, which is validated before confirm is enabled.

7.4 Audit context

Sale carries **cashier** and **shiftId**. The cashier is populated from the injected **OperatorContext**. The shift id is plumbing only and is always zero: **ShiftManager** and **ZReportDialog** exist but are not wired into the main window, so there is no live current-shift source. The column and field exist for when one is wired in.

8 Finance and accounting (Tier 4)

The finance subsystem is gated behind licence tier 4 and reached from the Finance menu.

8.1 General ledger

Ledger takes a **QSqlDatabase** by value and seeds a Kenyan chart of accounts on first initialisation. **postEntry()** enforces balanced debits and credits within a single transaction; an unbalanced entry cannot be written.

Range	Accounts
Assets (1xxx)	1000 Cash on Hand · 1010 M-Pesa / Mobile Money · 1020 Bank · 1100 Accounts Receivable · 1200 Inventory · 1300 VAT Input
Liabilities (2xxx)	2000 Accounts Payable · 2100 VAT Output · 2200 PAYE · 2210 SHIF · 2220 NSSF · 2230 Housing Levy · 2300 Net Pay
Equity (3xxx)	3000 Owner's Equity · 3100 Retained Earnings
Revenue (4xxx)	4000 Sales Revenue
Cost (5xxx)	5000 Cost of Goods Sold
Expenses (6xxx)	6000 Wages & Salaries · 6100 Rent · 6200 Utilities · 6300 Other

8.2 Automatic journal posting

Three pure builder functions in **salejournal** return balanced journal lines. Being pure, they are directly unit-tested.

Builder	Posting
buildSaleJournal	Debit tender accounts (cash 1000 / M-Pesa 1010 / card and bank 1020, resolved by <code>saleTenderAccount</code>) with non-cash settlement to AR 1100; credit 4000 Sales at net and 2100 VAT Output. Adds a debit 5000 COGS / credit 1200 Inventory pair when cost is known.
buildPurchaseJournal	Debit 1200 Inventory at net and 1300 VAT Input; credit 2000 Accounts Payable at gross.
buildExpenseJournal	Debit the mapped expense account (via <code>expenseAccount</code>); credit 1000 Cash.

Posting is invoked after the underlying transaction succeeds — sales from **CheckoutService**, purchases on goods receipt, expenses on entry. All three are **best-effort**: a posting failure is logged but never undoes the business transaction.

Posting lives in the UI layer for purchases and expenses

This keeps `database_test` isolated from ledger concerns, but it does mean the posting is skipped if those paths are ever driven programmatically rather than through their dialogs. Accounts receivable and payable accrue correctly, but settling them is still a manual journal.

8.3 VAT

Products carry a **tax_code** of Standard, Zero or Exempt. **splitInclusive()** backs VAT out of tax-inclusive prices using $\text{vat} = \text{gross} \times r / (1 + r)$. **computeVat3()** aggregates output VAT from sale items and input VAT from received purchase-order lines.

Transactional tables are the VAT source of truth

The VAT-3 return is computed from sales and purchase-order tables, not from the general ledger. This is deliberate: the return must reflect what was actually transacted, independently of whether best-effort GL posting succeeded.

The VAT rate is **read-only** in the VAT dialog. It is sourced from Settings → Tax and mirrored into **vat_config** at startup and on every settings change, so VAT calculations always use the rate customers are actually charged. This single-source arrangement fixed a latent divergence between the two.

eTIMS is defined as an interface with a stub implementation that logs and reports not-onboarded. Real KRA OSCU/VSCU integration requires device onboarding and is not implemented.

8.4 Payroll

computePayslip(gross, rates) is pure and tested. It implements the post-2024 Kenyan treatment in which NSSF, SHIF and the Housing Levy are deducted **before** PAYE is assessed.

Deduction	Basis
NSSF	6%, capped at 36,000 pensionable — maximum 2,160
SHIF	2.75%, minimum 300
Housing Levy	1.5%
PAYE	Progressive bands 10 / 25 / 30 / 32.5 / 35%, less personal relief of 2,400

A pay run stores payslips and posts one balanced journal entry: debit 6000 Wages; credit 2300 Net Pay, 2200 PAYE, 2210 SHIF, 2220 NSSF and 2230 Housing Levy. Rates persist in **payroll_config** and are editable.

Statutory rates require verification

The rates encoded here reflect the position at the time of implementation. Kenyan statutory rates change with each Finance Act — verify against the current Act before relying on payroll output. See docs/STATUTORY_RATES.md.

9 M-Pesa payment integration

9.1 Why a broker exists

A desktop till sits behind NAT and cannot receive Safaricom's callback. The public Cloudflare Worker is therefore the rendezvous point. Routing through it also keeps Daraja credentials server-side rather than on every shop PC.

9.2 Flow

- 1 The cashier enters the mobile-money amount and the customer's phone number, then presses *Send STK Prompt*.
- 2 **MpesaClient::requestPayment()** POSTs to **/pos/mpesa/stkpush**, authenticated with the till's licence key and device id — the same pair used for licence validation.
- 3 The Worker obtains a Daraja token and initiates the STK push, storing the request in **stk_requests** and returning the CheckoutRequestID.
- 4 Safaricom prompts the customer, who enters their M-Pesa PIN.
- 5 Daraja delivers the outcome to **/pos/mpesa/callback**, which updates the row's status, receipt and result description.
- 6 The till polls **/pos/mpesa/status** every 3 s, up to 25 times (75 s — deliberately longer than the prompt's own ~60 s life).
- 7 On success the receipt code populates the reference field and the sale can be confirmed; on failure or timeout the cashier may enter the code manually.

9.3 Why polling rather than push

Polling is a deliberate choice, not a shortcut. The worst case is three seconds of latency on a transaction where the customer is physically present entering a PIN, which itself takes 15–30 seconds. A persistent connection would buy nothing a cashier can perceive, while adding a socket that breaks on unreliable shop connectivity and needs reconnection logic. Polling is stateless and self-healing.

9.4 Paybill versus Buy Goods

Daraja treats these differently, and the distinction is easy to get wrong.

Transaction type	BusinessShortCode	PartyB
Paybill	Paybill number	The same number
Buy Goods (till)	Head Office / store number	The till number

BusinessShortCode is also what the **Password** hash is built from, so it must be the number the passkey was issued against — for Buy Goods that is the Head Office number, never the till. Set **MPESA_TILL_NUMBER** for Buy Goods; leaving it unset falls back to **MPESA_SHORTCODE**, which is correct for Paybill.

Sandbox will not catch a wrong pairing

Sandbox does not enforce the relationship between the Head Office number and the till, so a Buy Goods configuration can return **ResponseCode: 0** in sandbox and still fail in production. Confirm with a small live transaction at go-live.

9.5 Worker configuration

Variable	Purpose
MPESA_CONSUMER_KEY / _SECRET	Daraja application credentials (secrets).
MPESA_SHORTCODE	The shortcode the passkey belongs to. Paybill number, or the Head Office number for Buy Goods.
MPESA_PASSKEY	Daraja Lipa-na-M-Pesa passkey (secret).
MPESA_ENV	sandbox (default) or production.
MPESA_TXN_TYPE	paybill (default) or buygoods.
MPESA_TILL_NUMBER	Buy Goods only — sent as PartyB.

The C2B webhook is not the POS path

/webhook/mpesa handles licence purchases and assigns licence keys. POS sales travel exclusively through the STK path above. The two must not be confused.

10 Licensing and tiers

10.1 Activation

Activation is online-first: the client calls the Worker and falls back to local activation when the server is unreachable. This is deliberate — offline shops must never be locked out of their own till. Local activations are never heartbeat-checked.

Tier	Capability
1	POS Core
2	Adds inventory and customer depth
3	Reports, users, multi-till, messaging
4	Full ERP — general ledger, VAT, payroll

The server returns the tier on both activation and validation; the client applies it and caches it. During trial the effective tier is 4 so that everything can be evaluated. A full licence with no server-supplied tier defaults to 1 — the safe minimum. Feature gates call **checkLicenseTier()** at the menu-action level.

The licence check is a gate, not a security boundary

Client-side key validation and the tamper salt are obfuscation and format-checking. The server is the authority. This is documented honestly in the source rather than overstated. Note also that changing the tamper salt or the registry-hash formula after shipping will brick legitimate installations and requires a migration.

10.2 Credential storage

The SMTP password is encrypted at rest with Windows DPAPI (**CryptProtectData**, linked via crypt32) behind a **dpapi:v1:** marker. Settings load decrypts and migrates legacy plaintext values; save always encrypts.

11 Theming and the UI system

colorscheme.h is the single palette — base colours plus soft semantic surfaces for success, warning, error and info. **appstyle.cpp** generates one application-wide stylesheet from it, applied via **qApp->setStyleSheet** so that parentless dialogs such as the login window are themed too.

Styling is driven by dynamic properties rather than per-widget stylesheets:

Property	Values
kind	primary / danger / warning / info / secondary / tertiary — on buttons, labels and group boxes
role	amount / amountTotal / amountDiscount / totalTitle / banner / chip / statValue / statValueLg / dialogTitle / sectionTitle / hline
textScale	sm / md / lg / xl / 2xl — theme-independent sizing
bold	true

setStyleProperty() performs the unpolish/polish cycle needed for a runtime property change to repaint. Theme switching goes through **ThemeManager::setDark()** as the single source of truth; views that derive cell brushes

re-run their load to pick up new colours.

Deliberate exceptions to the theme

A handful of surfaces intentionally keep fixed colours: the Z-report's printable HTML (light is correct for print), the barcode preview canvas (white is required for scannability), and the WhatsApp brand-green test button.

11.1 Accessibility and touch

- High-DPI rounding policy is set to **PassThrough** before **QApplication** construction; card sizing is font-metric relative rather than fixed pixels.
- Stock severity is conveyed as **text** as well as colour (“CRITICAL — 3 left”, “OUT OF STOCK”), not colour alone.
- The product grid is keyboard navigable — arrow keys move, Enter adds, out-of-stock cards are skipped.
- The cart has tap-stepper columns flanking quantity, with 46 px rows for larger touch targets.

12 Testing

Seven CTest suites, all currently passing. The suite is built when **KEYNETIK_BUILD_TESTS=ON**.

Suite	Covers
carttotals_test	Tax, discount and rounding across 15 cases — including the characterisation test that documented the original double-rounding defect.
database_test	recordSale stock decrement, empty-item rejection, all-or-nothing rollback on insufficient stock, refund stock restoration, the double-refund guard, and money-migration persistence across reopen.
ledger_test	Balanced posting, account balances, trial balance.
payroll_test	Statutory computation across five cases, including an exact 50,000 gross breakdown.
vat_test	Inclusive VAT splitting and VAT-3 aggregation.
salejournal_test	11 cases across the sale, purchase and expense journal builders.
checkoutservice_test	finalizeSale with an injected operator and no session — cashier attribution, atomic stock decrement, audit callback, and insufficient-stock failure preserving stock.

12.1 What makes these tests possible

Two decisions carry the suite. First, the calculations live in pure functions, so they can be tested without a database or a widget. Second, **Database::configureForTesting()** repoints the connection at a private in-memory database and **initialize(false)** skips sample-data seeding, so each test starts from a known empty schema.

checkoutservice_test runs GUI-less by passing a null **ReceiptPrinter**; **finalizeSale** tolerates this deliberately, on the reasoning that a sale is already durably recorded and a terminal without a printer should not crash checkout. **ReceiptPrinter** is still linked because the symbol is referenced.

Two operational notes

Tests seed a category before inserting products, because **products.category** is an enforced foreign key. And QtTest output is swallowed by a PowerShell pipe — run the executable with **-o file.txt** when invoking a suite directly rather than through ctest.

12.2 What is not tested

The UI layer has no automated coverage. Converting the widgets to .ui files was assessed and recommended against — it is a large mechanical change with no functional benefit that does not improve testability, because the logic would remain welded to the widgets. The real fix is extracting view models, which is a separate and larger initiative.

13 Packaging and deployment

13.1 The desktop installer

A POST_BUILD step runs **windeployqt** into **windeployqt_stage/**, collecting Qt DLLs, platform plugins and SQL drivers beside a copy of the executable. **install()** ships that directory; CPack's NSIS generator wraps it with Start Menu and desktop shortcuts, matching uninstaller entries, the application icon and a run-on-finish option.

```
$env:PATH = "C:\Program Files (x86)\NSIS;" + $env:PATH
cpack --config "build\Desktop_Qt_6_11_0_MinGW_64_bit-Release\CPackConfig.cmake" \
-B "build\Desktop_Qt_6_11_0_MinGW_64_bit-Release"
```

Output is **KeynetikPOS-<version>-Setup.exe**, approximately 28 MB. The CPack block is conditional on **LICENSE.txt** existing; the icon is applied only if **resources/app.ico** is present.

Bump the version before every release

CPACK_PACKAGE_FILE_NAME derives from **APP_VERSION**, so packaging without bumping it silently overwrites the previous installer with an identically named file. Update both **APP_VERSION** and the **project()** version.

13.2 The Worker

```
cd license-server
wrangler d1 execute keynetik-license --remote --file=schema.sql
wrangler deploy
```

Wrangler requires Node 22 or later. Secrets are set with **wrangler secret put**; note that a shell pipe can append a newline to a secret, which has previously caused authentication failures — the admin token check trims for this reason.

13.3 Backups

backupIfDue() runs once per calendar day at startup: it checkpoints the WAL with **wal_checkpoint(TRUNCATE)**, copies the database file to the AppData backups directory with a timestamped name, and rotates to the newest ten. A manual **Backup Now** action is available under Settings, gated by the backup permission.

14 Known gaps and technical debt

Recorded honestly rather than omitted. Ordered by operational risk.

Area	Gap
M-Pesa reconciliation	If a callback arrives while the till is offline or closed, the sale is lost from the cashier's view although the customer was charged. The receipt exists in <code>stk_requests</code> but nothing reconciles it. A lookup screen by phone and amount would close this. Recommended before go-live.
Buy Goods verification	The Paybill/Buy Goods field semantics are implemented but have not been confirmed against a live Daraja endpoint; sandbox cannot validate the pairing.
Statutory rates	Payroll rates need checking against the current Finance Act.
eTIMS	Stub only. Real KRA OSCU/VSCU integration requires device onboarding.
Shift wiring	ShiftManager and ZReportDialog are orphaned — not reachable from the main window. <code>sales.shift_id</code> is therefore always zero.
AR / AP settlement	Receivables and payables accrue automatically, but settling them still requires a manual journal entry.
Inventory polling	InventoryManager reloads the whole product table every 60 seconds. In a single-process application there is no external stock writer, so this should be event-driven on sale, refund and restock, or the interval widened.
Reporting N+1	Reports fetch a product per sale line, although <code>sale_items</code> already snapshots the name and the lookup is largely unnecessary.
Stale documentation	<code>database.h</code> still describes the class as a singleton in its header comment, and <code>CODE_DOCUMENTATION.md</code> describes the pre-repository Database CRUD API. Both predate the refactors.
build-release.bat	Targets the wrong build directory and never produces an installer. Fix or delete.

A note on the two stale documents

The header comment on **database.h** and **CODE_DOCUMENTATION.md** both describe an architecture that no longer exists — the singleton was removed and the CRUD surface was replaced by repository accessors. They are actively misleading to a new reader and should be corrected or removed rather than left in place.

KeynetikPOS technical documentation, generated by `docs/generate_technical_documentation.py`. Structural claims verified against the source on 19 July 2026 (branch `feature/erp-tier4`, version 2.1.0).